

Programowanie obiektowe

Wykład 3

Hermetyzacja

Hermetyzacja w programowaniu obiektowym

Hermetyzacja (ang. *encapsulation*) – jeden z czterech filarów programowania obiektowego (obok abstrakcji, dziedziczenia i polimorfizmu).

Hermetyzacja polega na ukrywaniu wewnętrznych szczegółów implementacyjnych obiektu i udostępnianiu tylko tych elementów, które są niezbędne do interakcji z innymi częściami programu.

Założenia:

- **Oddzielenie interfejsu od implementacji** – użytkownik klasy powinien znać tylko dostępne metody (API) i nie musieć przejmować się szczegółami działania.
- **Ochrona danych** – wewnętrzne pola obiektów powinny być zabezpieczone przed bezpośrednią modyfikacją, co zapobiega błędom i niepożądanym zmianom stanu obiektu.
- **Zwiększenie czytelności i utrzymania kodu** – kod jest bardziej modularny, łatwiej go rozwijać i refaktoryzować bez wpływu na inne części systemu.

Interfejs vs implementacja

- **Interfejs** – sposób, w jaki użytkownik (człowiek lub inny program) może korzystać z systemu, bez zagłębiania się w jego wewnętrzne działanie.
- **Implementacja** – konkretne mechanizmy i kod, które wykonują operacje dostępne przez interfejs.

Interfejs i implementacja w różnych kontekstach:

- **Interfejs użytkownika (UI)** – to widoczna dla użytkownika część systemu, np. przyciski, formularze, menu.
Przykład: Strona internetowa ma przycisk „Wyślij wiadomość”, ale użytkownik nie widzi, co dzieje się po kliknięciu.
- **Interfejs programistyczny (API)** – zestaw funkcji i metod, które pozwalają innym programom korzystać z danego systemu bez znajomości jego szczegółów technicznych.
Przykład: System bankowy udostępnia funkcję `wypłaćKwotę()`, ale programista nie musi znać jej wewnętrznych obliczeń.

W obu przypadkach **interfejs** definiuje „co można zrobić”, a **implementacja** „jak to się dzieje”.

Interfejs i implementacja w programowaniu strukturalnym

W programowaniu strukturalnym (np. w C) również możemy mówić o interfejsie i implementacji.

- **Interfejs** to nagłówki funkcji (.h w C) – mówi, jakie operacje są dostępne.

```
// Plik nagłówkowy .h – deklaracja funkcji  
int add(int a, int b);
```

- **Implementacja** to kod w pliku .c – definiuje, jak funkcja działa.

```
// Plik .c – definicja funkcji  
int add(int a, int b) {  
    return a + b;  
}
```

- Kod wywołujący funkcję `add()` nie musi wiedzieć, jak działa jej wnętrze.

Interfejs i implementacja w programowaniu obiektowym

- **Interfejs** – sposób, w jaki obiekt udostępnia swoje funkcjonalności innym częściom programu. To zbiór metod i operacji, które obiekt oferuje, ale bez ujawniania, jak dokładnie działają.
 - **Definiuje sposób interakcji z obiektem** – użytkownik obiektu zna tylko dostępne metody.
 - **Ukrywa szczegóły wewnętrzne** – użytkownik nie wie, jak dokładnie realizowane są operacje.
 - **Zapewnia spójność kodu** – inne części programu mogą polegać na stabilnym interfejsie.
- **Implementacja** – konkretne rozwiązania techniczne wewnątrz obiektu, które realizują operacje dostępne przez interfejs. To szczegóły kodu, które pozostają ukryte przed użytkownikiem obiektu.
 - **Odpowiada za działanie metod i przetwarzanie danych** – definiuje rzeczywiste algorytmy i operacje.
 - **Może się zmieniać bez wpływu na interfejs** – można ulepszać kod bez modyfikowania sposobu korzystania z obiektu.
 - **Może wykorzystywać hermetyzację** – dane są przechowywane i przetwarzane wewnątrz obiektu, a dostęp do nich odbywa się przez kontrolowane metody.

Interfejs

Dostępne operacje skarbonki.

```
class PiggyBank
{
    double totalAmount;

    PiggyBank() // ●
    {
        totalAmount = 0;
    }

    void Deposit(double amount) // ●
    {
        totalAmount += amount;
    }

    double GetTotalAmount() // ●
    {
        return totalAmount;
    }
}
```

Implementacja

Ukryta logika.

```
class PiggyBank
{
    double totalAmount; // ■

    PiggyBank()
    {
        totalAmount = 0; // ■
    }

    void Deposit(double amount)
    {
        totalAmount += amount; // ■
    }

    double GetTotalAmount()
    {
        return totalAmount; // ■
    }
}
```

Zmiana interfejsu a zmiana implementacji

- **Modyfikowanie interfejsu** (czyli zmiana sposobu, w jaki obiekt jest używany):
 - **Może powodować konieczność zmian w całym kodzie** – jeśli zmienimy dostępne metody, inne klasy korzystające z obiektu mogą przestać działać.
 - **Powinno być dobrze przemyślane** – interfejsy powinny być stabilne, aby unikać częstych zmian.
 - Dodawanie nowych metod jest bezpieczniejsze niż modyfikowanie istniejących.
- **Modyfikowanie implementacji** (czyli zmiana działania metod, ale bez zmiany sposobu ich wywoływania):
 - **Nie wpływa na inne części kodu** – jeśli interfejs pozostaje taki sam, użytkownicy obiektu nie muszą nic zmieniać.
 - **Pozwala na optymalizację lub ulepszenie działania** – można poprawić wydajność lub dodać nowe funkcjonalności bez zmiany interakcji z obiektem.
 - Może wykorzystywać różne algorytmy, np. zmiana sposobu przetwarzania danych w bazie bez wpływu na użytkownika klasy.

Zmiana interfejsu

Niebezpieczna

Zmiana interfejsu może sprawić, że kod klienta przestanie działać:

```
class PiggyBank
{
    double totalAmount;
    PiggyBank() {
        totalAmount = 0;
    }

    void Deposit(double coinValue, int count) {
        totalAmount += coinValue * count;
    }

    double GetTotalAmount() {
        return totalAmount;
    }
}

// Kod klienta (Main())
PiggyBank piggyBank = new PiggyBank();
piggyBank.Deposit(2.5); // ❌
```


Zmiana implementacji

Bezpieczna

Zmiana sposobu przechowywania monet nie wpływa na kod klienta.

```
class PiggyBank
{
    double[] coins;
    int count;

    PiggyBank() {
        coins = new double[100];
        count = 0;
    }
    void Deposit(double amount) {
        coins[count++] = amount;
    }
    double GetTotalAmount() {
        double total = 0;
        for (int i = 0; i < count; i++)
            total += coins[i];
        return total;
    }
}

// Kod klienta (Main())
PiggyBank piggyBank = new PiggyBank();
piggyBank.Deposit(2.5); // ✓
```

Modyfikatory dostępu

- **Poziomy dostępu** - określają, kto może korzystać z pól i metod klasy w kodzie.
- **Modyfikatory dostępu** - słowa kluczowe używane do określenia poziomu dostępu do pól, metod i klas.

Główne poziomy dostępu

- **Prywatny (private)** - dostęp tylko wewnątrz tej samej klasy, brak dostępu z zewnątrz.
 - Najbardziej restrykcyjny poziom dostępu.
 - Używany do ukrycia wewnętrznych zmiennych i metod.
- **Chroniony (protected)** - dostęp w tej samej klasie i jej podklasach (wrócimy do niego przy okazji rozmowy o *dziedziczeniu*).
- **Publiczny (public)** - dostęp bez ograniczeń, także z innych części programu.
 - Stosowany do głównej funkcjonalności obiektów, np. metod wywoływanych z innych części systemu.
- **Pakietowy** - dostęp w obrębie tego samego pakietu (Java) lub assembly (C#).
 - Używany, gdy chcemy udostępnić metody w ramach jednej części systemu, ale nie na zewnątrz.

Dostęp publiczny

```
class Point {  
    public Point(double x, double y) { ... }  
    public void Move(double dx, double dy) { ... }  
    public void Display() { ... }  
}
```

Z publicznych składowych może korzystać każdy:

```
Point point = new Point(2.0, 3.0);  
point.Display();  
  
point.Move(1.5, -2.0);  
point.Display(); // Powinno wyświetlić (3.5, 1.0)
```

Dostęp prywatny

```
class Point {  
    private double x;  
    private double y;  
    public Point(double x, double y) { this.x = x; this.y = y; }  
    public void Move(double dx, double dy) { x += dx; y += dy; }  
    public void Display() { Console.WriteLine($"Point is at ({x}, {y})"); }  
}
```

Z prywatnych składowych klasy może korzystać tylko kod umieszczony tej klasie.

```
Point point = new Point(2.0, 3.0);  
Console.WriteLine(point.x); // ✗  
point.y = 0.0; // ✗
```

```
class Point {  
    ...  
    public void Move(double dx, double dy)  
    {  
        this.x += dx; // ✓  
        this.y += dy; // ✓  
    }  
}
```

Hermetyzacja klasowa

W językach C++, Java i C# występuje hermetyzacja na poziomie klasy.

```
Point point = new Point(2.0, 3.0);  
Console.WriteLine(point.x); // ✗  
point.y = 0.0; // ✗
```

Kod klienta nie ma dostępu do składowych prywatnych.

Natomiast w **kodzie klasy** mamy dostęp do składowych prywatnych **innego obiektu** tej klasy.

```
class Point {  
    ...  
    public double DistanceTo(Point other)  
    {  
        double dx = other.x - this.x; // ✓  
        double dy = other.y - this.y; // ✓  
        return Math.Sqrt(dx * dx + dy * dy);  
    }  
}
```

Publiczny interfejs, prywatna implementacja

Zwykle pola klasy są prywatne, a metody – publiczne.

Mogą istnieć **prywatne metody** – nienależące do interfejsu.

```
class PasswordManager {  
    private string password; // Przechowywane hasło  
  
    // ● Publiczna metoda do ustawiania nowego hasła  
    public void SetPassword(string newPassword) {  
        if (IsValidPassword(newPassword)) {  
            password = newPassword;  
            Console.WriteLine("Password has been updated.");  
        }  
        else {  
            Console.WriteLine("Invalid password. Must be at least 6 characters long.");  
        }  
    }  
  
    // ■ Prywatna metoda sprawdzająca poprawność hasła  
    private bool IsValidPassword(string newPassword) {  
        return newPassword.Length >= 6;  
    }  
}
```

Publiczne pola należą do złych praktyk.

Modyfikatory dostępu w C++

Hermetyzacja w C++ opiera się na sekcjach `public`, `private` i `protected`.

```
class Point {  
    // Domyślnie pierwszą otwartą sekcją jest sekcja prywatna  
    double x;  
    double y;  
public:  
    // Sekcja publiczna  
    Point(double x, double y): x(x), y(y) { }  
    void move(double dx, double dy) {  
        x += dx;  
        y += dy;  
    }  
    void display() {  
        cout << "(" << x << ", " << y << ")" << endl;  
    }  
    // Sekcje można otwierać wielokrotnie  
private:  
    string color;  
public:  
    void setColor(string color) {  
        this.color = color;  
    }  
};
```

Hermetyzacja w C++ - struct

- W C++ zachowano słowo kluczowe `struct` (kompatybilność wsteczna z C), ale działa ono jak `class` - możemy definiować metody, konstruktory, destruktory w strukturach.
- Jedyną różnicą jest domyślny poziom dostępu - w **strukturze** jest on **publiczny**.
- Struktury przeważnie używane są do tworzenia prostych obiektów bez logiki.

```
class Point {  
    double x; // domyślnie prywatne  
    double y;  
};
```

```
struct Point {  
    double x; // domyślnie publiczne  
    double y;  
};
```


Hermetyzacja w C++ - friend

- Słowo kluczowe `friend` pozwala na **złamanie hermetyzacji**.
- Klasa może dać dostęp do **swoich** prywatnych składowych innej klasie lub funkcji.

```
class Point;  
  
class Physics {  
public:  
    void move(Point&);  
};  
  
class Point{  
    double x;  
    double y;  
    friend class Renderer;  
    friend void display(const Point&);  
    friend void Physics::move(Point&);  
};
```

```
class Renderer {  
public:  
    void render(const Point& point) {  
        cout << "(" << point.x << ", ";  
        cout << point.y << ")" << endl;  
    }  
};  
  
void display(const Point& point) {  
    cout << "(" << point.x << ", ";  
    cout << point.y << ")" << endl;  
}  
  
void Physics::move(Point& point) {  
    point.x += ...;  
    point.y += ...;  
}
```

Ta kolejność deklaracji jest konieczna: `Point` musi znać klasę `Physics` aby zadeklarować zaprzyjaźnienie z jej metodą, natomiast `Physics` musi znać klasę `Point` aby użyć jej jako parametru. Definicja metody `Physics::move()` musi znaleźć się za pełną definicją klasy `Point`, bo odwołujemy się do jej pól.

Modyfikatory dostępu w Javie

W Javie poziom dostępu jest określany osobno dla każdej składowej.

```
class Point {  
    private double x;  
    private double y;  
    public Point(double x, double y) { ... }  
    public void move(double dx, double dy) { ... }  
    void display() { ... }  
}
```

- Poza poziomami `public`, `private` i `protected` mamy dostęp domyślny – pakietowy.
 - Brak modyfikatora dostępu oznacza dostęp **pakietowy**.
 - Takie składowe są dostępne jedynie wewnątrz *pakietu*.
- Modyfikator `protected` ogranicza dostęp jedynie na zewnątrz pakietu.

Modyfikatory dostępu w C#

W C# użycie modyfikatorów dostępu jest takie jak w Javie – określa się je osobno dla każdej składowej.

```
class Point {  
    private double x;  
    private double y;  
    public Point(double x, double y) { ... }  
    public void Move(double dx, double dy) { ... }  
    internal void Display() { ... }  
}
```

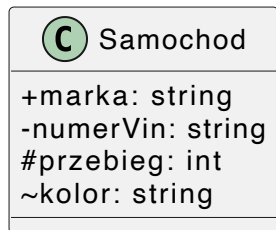
- Poza `public`, `private` i `protected` istnieje dostęp `internal`:
 - Składowe dostępne wewnątrz tego samego *assembly* (modułu wykonywalnego, np. plik **.exe** lub **.dll**).
 - Niedostępne na zewnątrz *assembly*.
- `protected internal` – publiczne wewnątrz *assembly*, chronione na zewnątrz.
- `private protected` – chroniony wewnątrz *assembly*, prywatny na zewnątrz.
- Dostęp **domyślny** (gdy nie podamy modyfikatora dostępu) – **prywatny**.

Modyfikatory dostępu w UML

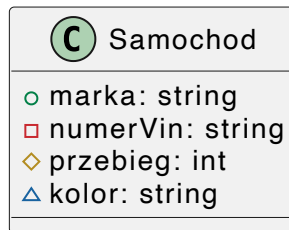
W UML modyfikatory dostępu są oznaczane symbolami przed nazwami atrybutów i metod.

- **+** (public) – dostępny dla wszystkich
- **-** (private) – dostępny tylko w tej samej klasie
- **#** (protected) – dostępny w tej klasie i klasach dziedziczących
- **~** (package / default) – dostępny tylko w tym samym pakiecie

Narzędzia (np. *PlantUML*) mogą zastępować to ikonami.



Oznaczenie symbolami + - # ~



Oznaczenie ikonami

```
class Samochod {  
    public string marka;           // Dostępny dla wszystkich  
    private string numerVin;       // Unikalny numer pojazdu (dostęp tylko wewnątrz klasy)  
    protected int przebieg;        // Może być modyfikowany przez podklasy  
    internal string kolor;         // Dostępny tylko w obrębie tego samego assembly  
}
```

Hermetyzacja a dostęp do stanu

Zgodnie z regułą **hermetyzacji**, klient nie ma bezpośredniego dostępu do (*prywatnego*) stanu obiektu. Może go uzyskać przy pomocy metod.

Podstawowy sposób to metody logiki. **Dobłą praktyką** jest dzielenie ich na pytania i żądania:

- **Pytania (*queries*)** – służą do odczytu danych i nie zmieniają stanu obiektu.
- **Żądania (*commands*)** – ich zadaniem jest wykonanie akcji, służą do modyfikowania obiektu, ale nie zwracają wartości.

```
class Stack
{
    public void Push(double value)
    {
        items[++top] = value;
    }
    public double Peek()
    {
        return items[top];
    }
    ...
}
```

C Stack
□ items: double[] □ top: int
● Push(value) ● Pop() ● Peek() ● IsEmpty()

Ochrona danych przed dostępem

Pola klasy oznaczone jako **prywatne** (`private`) nie mogą być modyfikowane bezpośrednio z zewnątrz.

- Zapobiega to przypadkowemu nadpisaniu lub niepożądaney zmianie wartości kluczowych dla działania obiektu.

```
class KontoBankowe {  
    // Prywatne pole, chronione przed zmianą z zewnątrz  
    private double saldo;  
  
    public double pobierzSaldo() {  
        // Kontrolowany dostęp do wartości  
        return saldo;  
    }  
}
```

 KontoBankowe
 saldo: double
 pobierzSaldo(): double

- Bezpośrednia modyfikacja pola `saldo` nie jest możliwa, co chroni przed błędami.

```
KontoBankowe konto = new KontoBankowe();  
konto.saldo = 1000; // ❌ Pole prywatne – brak dostępu
```

Kontrola nad sposobem zmiany danych

Dane są modyfikowane wyłącznie przez metody klasy.

- Metody mogą odpowiadać za walidację, np. nie pozwalać na dokonanie ujemnej wpłaty.
- Zapewniają spójność i integralność obiektu, nie pozwalając na wprowadzenie niepoprawnego stanu, np. ujemnego salda.

```
class Konto {  
    private double saldo;  
  
    public void Wplac(double kwota) {  
        if (kwota > 0) saldo += kwota;  
    }  
  
    public bool Wyplac(double kwota) {  
        if (kwota > 0 && kwota <= saldo) {  
            saldo -= kwota;  
            return true;  
        }  
        return false;  
    }  
}
```

 Konto
□ saldo: double
● Wpłać(kwota)
● Wypłać(kwota)

- Klasa sama zarządza swoimi danymi.

Metody dostępne

Jeśli chcemy dać bezpośredni dostęp do pól, dobrą praktyką jest użycie **metod dostępowych**.

Metody dostępne umożliwiają kontrolowany dostęp do pól klasy, zazwyczaj oznaczonych jako `private`.

- **Gettery (*get methods*)** – zwracają wartość prywatnego pola, pozwalając na jego odczyt bez możliwości bezpośredniej modyfikacji.
- **Settery (*set methods*)** – umożliwiają zmianę wartości prywatnego pola.

```
class Point {  
    private double x, y;  
  
    public double getX() { return x; }  
    public double getY() { return y; }  
  
    public void setX(double x) { this.x = x; }  
    public void setY(double y) { this.y = y; }  
}
```

C Point	
□ x: double	
□ y: double	
● getX(): double	
● getY(): double	
● setX(x)	
● setY(y)	

- Inne funkcjonujące pojęcia to **akcesory (*accessors*)** i **mutatory (*mutators*)** – pozwalające odpowiednio na odczyt i zmianę wartości pól.

Metody dostępne

- Zachowują hermetyzację – chronią dane przed bezpośrednim dostępem, zapobiegając przypadkowym zmianom i zapewniając spójność stanu obiektu.

```
public class User {  
    private String userId;    // Identyfikator użytkownika (tylko do odczytu)  
    private String password; // Hasło użytkownika (tylko do zapisu)  
  
    public User(String userId) {  
        this.userId = userId;  
    }  
  
    public String getUserId() {  
        return userId;  
    }  
  
    public void setPassword(String password) {  
        if (password != null && password.length() >= 6) {  
            this.password = password;  
        } else {  
            return;  
        }  
    }  
}
```

- Możliwość dodania logiki – mogą zawierać dodatkową walidację, np. sprawdzenie poprawności danych przy ustawianiu wartości.

Zadania metod dostępowych

- Ich użycie jest na tyle powszechne, że IDE mogą dodawać je automatycznie.
- Ich nazwy (`setNazwa()` , `getNazwa()`) to jedynie kwestia konwencji.
- Poza dostępem do danych, mogą mieć dodatkowe zadania.

```
public class Range {  
    private int min; // Minimalna wartość zakresu  
    private int max; // Maksymalna wartość zakresu  
  
    // Setter dla wartości minimalnej  
    public void setMin(int min) {  
        if (min > this.max) {  
            // Zmiana min, zxmienia również max  
            this.min = this.max;  
            this.max = min;  
        } else {  
            this.min = min;  
        }  
        // Odświeżenie widoku  
        App.getMainWindow.refresh();  
    }  
  
    // Setter dla wartości maksymalnej  
    public void setMax(int max) { ... }  
}
```

Metody dostępne – dobre i złe praktyki

- Nie jest zalecane modyfikowanie pola do którego sięgamy w getterze.

```
public class Counter {  
    private int value;  
  
    public int getValue() { // ✗ Getter zmienia stan  
        return value++;  
    }  
}
```

Gettery nie powinny mieć **efektów ubocznych**.

Metody dostępne – dobre i złe praktyki

- W niektórych przypadkach modyfikowanie stanu jest dozwolone, np.:
 - Opóźniona inicjalizacja (*Lazy Loading*) – ładujemy zasoby (np. dane z pliku) dopiero wtedy, gdy są potrzebne.
 - Cache’owanie wyniku – jeśli obliczenie wartości jest kosztowne, getter może zapisywać wynik w polu, aby nie wykonywać obliczeń za każdym razem.

```
public class ExpensiveCalculation {  
    private Double cachedResult;  
  
    public double getComplexCalculation() {  
        if (cachedResult == null) {  
            cachedResult = Math.random() * 100; // Symulacja kosztownego obliczenia  
        }  
        return cachedResult;  
    }  
}
```

Właściwości (*properties*) w C#

W **C#**, zamiast klasycznych getterów i setterów, używa się **właściwości (*properties*)**, które **łączą zalety metod dostępowych (`get` , `set`) z prostą składnią pól**.

Klasyczne gettery wymagają wywoływania metod.

```
class User {  
    private string name;  
  
    public string Name {  
        get {  
            return name;  
        }  
        set {  
            name = value;  
        }  
    }  
}  
  
// Main()  
User user = new User();  
user.Name = "Kowalski"; // Ustawienie wartości  
Console.WriteLine(user.Name); // Pobranie wartości
```

- Właściwości można ustawiać i odczytywać tak jak pola, ale zachowując kontrolę jak w getterach/setterach.

Właściwości (*properties*) w C#

Właściwości (`properties`) łączą logikę getterów i setterów w jednym elemencie składniowym.

Tak jak w klasycznych getterach/setterach, właściwości pozwalają na:

- Walidację lub inne operacje wewnątrz `get` i `set` .
- Definiowanie właściwości tylko do odczytu lub tylko do zapisu.

```
class User {  
    private string userId;  
    private string password;  
  
    public User(string userId) {  
        this.userId = userId;  
    }  
  
    public string UserId {  
        get { return userId; } // tylko getter  
    }  
  
    public string Password {  
        set { password = value; }  
        private get { return password; } // prywatny getter  
    }  
}
```

Automatyczne właściwości (auto-properties)

Jeśli właściwość nie wymaga dodatkowej logiki, można użyć **automatycznych właściwości**.

```
class User {  
    public string Name { get; set; } // Automatycznie zarządzane pole  
    public int Id { get; private set; } // get i set mogą mieć inny modyfikator dostępu  
}
```

- Nie trzeba definiować prywatnego pola – C# robi to automatycznie.
- Właściwości (nie tylko automatyczne) pozwalają na ustawianie wartości w momencie tworzenia obiektu (*object initializer*) – alternatywa dla konstruktorów.

```
User user = new User { Name = "Jan Kowalski" }; // Wywoła się konstruktor domyślny  
Console.WriteLine(user.Name);
```

- Od C# 9 można użyć **init** **zamiast** **set**, aby wartość była ustawiana tylko podczas inicjalizacji.

```
class User {  
    public string Name { get; init; } // Można ustawić tylko raz  
}
```

Hermetyzacja – podsumowanie

- **Ochrona danych** – niepożądany dostęp jest blokowany.
- **Kontrola zmian** – dane zmieniają się tylko w określony sposób.
- **Spójność obiektu** – zawsze poprawny stan.
- **Ukrycie implementacji** – użytkownik klasy nie musi znać jej działania.
- **Łatwiejsza refaktoryzacja** – zmiany wewnętrzne nie wpływają na resztę systemu.
- **Lepsza testowalność** – kod staje się bardziej modularny.

Uwaga: bezpieczeństwo

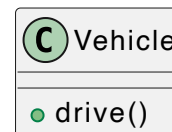
- Gdy mowa o bezpieczeństwie, chodzi o **zapobieganie błędom programisty** a nie **ochronę przed złośliwym działaniem**
- Nie chroni przed świadomymi atakami – kod działający w tym samym procesie może obejść hermetyzację np. przez refleksję (Reflection w C# i Javie) lub bezpośrednie manipulowanie pamięcią (C++).

Abstrakcja a hermetyzacja

Porównanie pojęć

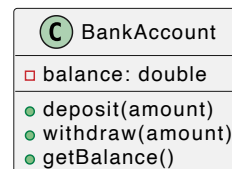
1. **Abstrakcja** – skupia się na tym co *robi obiekt*

- Ukrywanie zbędnych szczegółów implementacyjnych, pokazując tylko istotne aspekty.
- Upraszcza interakcję z obiektem poprzez definiowanie interfejsów.
- Redukuje złożoność kodu, ułatwia projektowanie systemów.
- Przykład: `Pojazd` z metodą `jedź()`, bez wskazania, jak działa silnik.



2. **Hermetyzacja** – określa *jak chronić dane* wewnątrz obiektu

- Ograniczanie dostępu do wewnętrznych danych obiektu i kontrolowanie ich modyfikacji.
- Chroni dane przed nieautoryzowanym dostępem i przypadkowymi zmianami.
- Poprawia bezpieczeństwo i integralność danych.
- Przykład: `KontoBankowe`, gdzie `saldo` jest `private`, a dostęp do niego odbywa się przez metody `wpłacić()` i `wypłacić()`.



Hermetyzacja – dobre praktyki

1. **Używaj `private` dla pól** – bezpośredni dostęp do zmiennych klasy powinien być ograniczony, a manipulacja nimi powinna odbywać się przez metody.
 - *Typowy błąd: udostępnianie pól jako `public`.*
2. **Udostępniaj dane przez metody (getter i setter)** – jeśli konieczny jest dostęp do pól, kontroluj go za pomocą metod zamiast udostępniać je bezpośrednio.
 - *Typowy błąd: brak kontroli w setterach.*
3. **Unikaj nadmiernej ekspozycji (`public`)** – nie wszystkie metody muszą być dostępne na zewnątrz klasy; domyślnie stosuj jak najniższy poziom dostępu.
4. **Unikaj setterów dla pól, które nie powinny się zmieniać** – jeśli wartość ma być stała po inicjalizacji, nie udostępniaj metody do jej zmiany.
 - Pełen zestaw setterów i getterów do każdego pola **bywa** zasadny, ale **nie zawsze**.

Metoda `toString()`/`ToString()`

Java i **C#** oferują standardowy sposób udostępnienia stanu obiektu bez naruszania hermetyzacji.

Jej zadaniem jest przekonwertowanie obiektu na jego reprezentację tekstową.

Jest ona automatycznie wywoływana np. podczas wypisywania obiektu na ekran lub konkatencji ze stringiem.

```
// Java
Point p = new Point(3, 4);
System.out.println(p);
```

```
// C#
Point p = new Point(3, 4);
Console.WriteLine(p);
```

Ale możemy też wywołać ją ręcznie.

```
System.out.println(p.toString());
```

```
Console.WriteLine(p.ToString());
```

Jej domyślna implementacja zwraca pełną nazwę klasy, a w Javie dodatkowo jej *hashCode*.

W Javie:

```
package.Point@1a2b3c
```

W C#:

```
Namespace.Point
```

Nadpisywanie metody `toString()` / `ToString()`

Przeważnie nadpisujemy metodę `toString()` / `ToString()` aby zwrócić bardziej kompletną **tekstową reprezentację stanu obiektu**.

```
// Java
class Point {
    private double x, y;

    @Override
    public String toString() {
        return "(" + x + ", " + y + ")";
    }
}
```

```
// C#
class Point {
    private double x, y;

    public override string ToString() {
        return $"({x}, {y})";
    }
}
```

Wyjście programu:

(3, 4)

- Modyfikator `override` w C# jest wymagany, aby oznaczyć że nadpisujemy domyślną wersję metody.
- Adnotacja `@Override` w Javie pełni podobną rolę, ale nie jest obowiązkowa - zapobiega jednak błędom i zgłaszaniu ostrzeżeń przez kompilator.

Wartości stałe w C#

Każdy język ma swoje sposoby na **oznaczenie wartości jako stałej**.

`const`

- Definiuje stałe wartości **czasu kompilacji** (ang. *compile-time constants*).
- Musi być zainicjalizowane **w momencie deklaracji**.
- Może być używane tylko dla **typów podstawowych i** `string` (np. `int`, `double`, `string`).

```
class Constants {  
    public const double Pi = 3.14159; // Stała kompilacyjna  
}
```

- **Efekt:** Nie można zmienić wartości `Pi` po skompilowaniu kodu.

Ograniczenia `const` :

- Nie może być używane dla obiektów lub list.

Wartości stałe w C#

readonly

- Pozwala na przypisanie wartości tylko raz – **w konstruktorze lub w deklaracji**.
- Działa **w czasie uruchomienia (*runtime*)**, nie w czasie kompilacji (*compile-time*).
- Może być używane dla **obiektów, struktur i tablic**.

```
class BankAccount
{
    public readonly string AccountNumber; // Numer konta, ustawiany raz i niezmienny

    public BankAccount(string accountNumber)
    {
        AccountNumber = accountNumber; // Można przypisać tylko w konstruktorze
    }
}
```

- **Efekt:** `readonly` pozwala na ustawienie wartości w konstruktorze, ale nie można jej zmienić później.

Kiedy używać `readonly` zamiast `const` ?

- Jeśli wartość nie jest znana w czasie kompilacji.
- Jeśli chcesz przypisać obiekt i nie zmieniać jego referencji.

Wartości stałe w C#

readonly

Modyfikator `readonly` chroni referencję, ale nie dane w obiekcie.

- Nie możemy zastąpić obiektu innym.
- Ale możemy zmienić stan obiektu.

```
class BankAccount {  
    public readonly Customer AccountHolder;  
    public BankAccount(Customer accountHolder) {  
        AccountHolder = accountHolder; // Można przypisać tylko raz  
    }  
  
    public void ChangeCustomer(Customer newAccountHolder) {  
        AccountHolder = newAccountHolder; // ❌ Próba zmiany referencji – BŁĄD  
    }  
  
    public void ChangeCustomerName(string newName) {  
        AccountHolder.Name = newName; // ✅ Można zmienić stan obiektu  
    }  
}
```

Wartości stałe w Javie

final

- Działa **podobnie jak** `readonly` **w C#** – pozwala przypisać wartość tylko raz.
- Można używać dla **zmiennych, metod i klas**.
- Działa **w czasie uruchomienia (runtime)**.

```
class Configuration {  
    public final String appName;  
  
    public Configuration() {  
        this.appName = "My Application"; // Można przypisać tylko raz  
    }  
}
```

- **Efekt:** `final` działa jak `readonly` w C# – pozwala na przypisanie wartości w konstruktorze.

Wartości stałe w C++

const

- Stała wartość, przypisana w czasie uruchomienia (*runtime*).
- Nie może zostać zmieniona po przypisaniu.
- Może być używane dla **zmiennych, pól klasy, wskaźników i referencji**.
- W przypadku pól klasy wymaga **konstruktora inicjalizującego**.

```
class BankAccount {  
public:  
    const int accountNumber; // Stała wartość przypisana raz  
  
    // Konstruktor przypisujący wartość do stałego pola  
    BankAccount(int number) : accountNumber(number) { }  
};
```

- **Efekt:** Każda instancja `BankAccount` może mieć inne `accountNumber`, ale po przypisaniu nie można go zmienić.

Wartości stałe w C++

constexpr

- Stała obliczana w czasie kompilacji (*compile-time*).
- Wartość musi być znana w czasie kompilacji.
- Może być używane do **stałych funkcji** (`constexpr function`) i **wyrażeń**.

```
constexpr int square(int x) {  
    return x * x;  
}  
  
int main() {  
    constexpr int result = square(5); // Obliczane w czasie kompilacji  
}
```

- **Efekt:** `square(5)` zostanie obliczone podczas kompilacji i zastąpione wartością `25`.

Wartości stałe w C++

`const` i wskaźniki

Wskaźniki mogą być `const` na różne sposoby:

- `const int* ptr;` - Wartość, na którą wskazuje wskaźnik, jest stała (nie można zmieniać `*ptr`, ale można zmieniać `ptr`).
- `int* const ptr;` - Wskaźnik jest stały (nie można zmienić `ptr`, ale można modyfikować wartość `*ptr`).
- `const int* const ptr;` - Ani wskaźnik, ani wartość, na którą wskazuje, nie mogą być zmienione.

```
int value = 10;
const int* ptr1 = &value; // Można zmieniać wskaźnik, ale nie wartość
int* const ptr2 = &value; // Można zmieniać wartość, ale nie wskaźnik
const int* const ptr3 = &value; // Ani wskaźnik, ani wartość nie mogą być zmienione
```

Grupowanie klas

Pakiet (`package`) w **Java** i przestrzeń nazw (`namespace`) w **C#** i **C++** to sposób organizacji klas.

- Pozwala na logiczne organizowanie kodu przez grupowanie powiązanych klas.
- Zapobiega konfliktom nazw - różne pakiety/przestrzenie nazw mogą zawierać klasy o tej samej nazwie.
- Zapewnia kontrola dostępu ograniczając dostępność klas i metod.
- Ułatwia zarządzanie dużymi projektami.

Pakiety (package) w Javie

- Pakiet w **Java** deklaruje się na początku pliku `*.java` , przed definicją klasy.

```
// Account.java
package com.bank;

public class Account {
    // ...
}
```

- Pakiety tworzą hierarchiczną strukturę mającą odzwierciedlenie w strukturze katalogów, w których umieszczamy pliki klas.
- Plik `Account.java` powinien znajdować się w katalogu:

```
/src/com/bank/Account.java
```

- Najczęściej stosuje się konwencję odwróconej domeny internetowej firmy czy organizacji.
- Jeśli nie zadeklarujemy pakietu, klasa znajduje się w **pakiecie domyślnym** – w większych projektach **nie jest to zalecane**.

Importowanie pakietu (`import`) w Javie

- Jeśli chcemy użyć klasy `Account` w klasie z innego pakietu, powinniśmy ją zaimportować:

```
// pakiet domyślny
import com.bank.Account;

public class Main {
    public static void main(String[] args) {
        Account acc = new Account(1000);
        System.out.println(acc.getBalance());
    }
}
```

- Inne rozwiązanie to użycie pełnej nazwy klasy:

```
com.bank.Account acc = new com.bank.Account(1000);
```

- Zamiast importowania konkretnej klasy, możemy zaimportować wszystkie klasy z pakietu (`*`).

```
import com.bank.*;
```

- **Nie importuje to podpakietów!** – Jeśli istnieje `com.bank.utils` , trzeba je zaimportować osobno.

Pakiety w Javie a hermetyzacja

Domyślny poziom dostępu (bez modyfikatora) oznacza poziom pakietowy: **publiczny** wewnątrz pakietu, a **prywatny** na zewnątrz.

- Może być stosowany zarówno do składowych jak i klas.
- W jednym pliku może znajdować się tylko jedna klasa publiczna.

```
package com.bank;
public class BankAccount {
    double balance;

    public BankAccount(double balance) {
        this.balance = balance;
    }
}
class Transaction {
    void deposit(BankAccount account) {
        // ✓ Dostęp pakietowy
        account.balance += 1000;
    }
}
```

```
package app;

import com.bank.*;

public class Main {
    public static void main(String[] args) {
        BankAccount acc = new BankAccount(0);
        // ✗ Niedostępne poza pakietem!
        acc.balance += 1000;

        // ✗ Nie można utworzyć instancji!
        Transaction tx = new Transaction();
    }
}
```

Przestrzenie nazw (namespace) w C#

Pozwalają na grupowanie klas.

```
namespace Bank {  
    namespace Accounts {  
        class Account {  
            ...  
        }  
    }  
}  
  
namespace Services {  
    class LoanService {  
        ...  
    }  
    class CreditService {  
        ...  
    }  
}
```

- Przestrzenie nazw można dowolnie zagnieżdżać.

Użycie przestrzeni nazw w C# (using)

Aby uniknąć konieczności podawania pełnej ścieżki do klasy, należy użyć `using`.

```
using Bank.Services;

namespace MyApplication
{
    class Program
    {
        static void Main()
        {
            CreditService cs = new CreditService();
        }
    }
}
```

- Bez `using` należałoby pisać `Bank.Services.CreditService`.

Unikanie konfliktów w przestrzeniach nazw w C#

- Jeśli dwie przestrzenie zawierają klasy o tej samej nazwie:

```
namespace Bank {  
    namespace Accounts {  
        class Account {  
            ...  
        }  
    }  
}
```

```
namespace Mail {  
    namespace Services {  
        class Account {  
            ...  
        }  
    }  
}
```

- Można użyć aliasu:

```
using Banking = Bank.Accounts;  
using Mail.Services;  
  
// Tworzymy konto z banku (namespace Bank.Accounts)  
Banking.Account acc1= new Banking.Account();  
  
// Tworzymy konto pocztowe (z namespace Mail.Services)  
Account acc1= new Account();
```

Przestrzeń nazw (namespace) w C++

- Deklaruje się je podobnie jak w C#.

```
#include <iostream>

namespace Bank {
    class Account {
        // ...
    };
}

int main() {
    Bank::Account acc("Kowalski", 1000); // Dostęp do klasy w przestrzeni nazw
}
```

- Podając pełne nazwy, używa się operatora zakresu `::`.
- Skrócić zapis możemy deklarując użycie.

```
// Użycie pojedynczej klasy
using Bank::Account;
```

```
// Użycie całej przestrzeni nazw
using namespace Bank;
// Uwaga: Może powodować konflikty
```

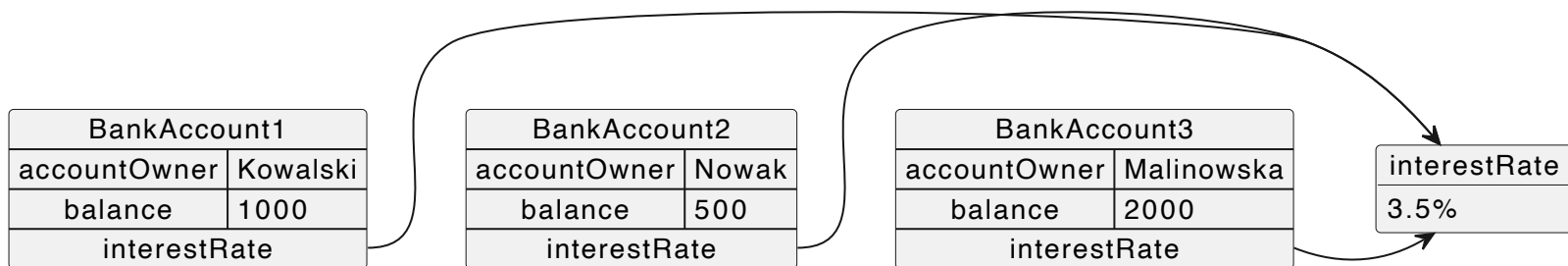
Składowe statyczne

Składowe statyczne (`static`) należą **do klasy, a nie do konkretnej instancji obiektu**. Oznacza to, że są współdzielone między wszystkimi instancjami danej klasy.

■ Pola statyczne

- Przechowują wartość wspólną dla wszystkich instancji klasy.
- Nie są związane z konkretnym obiektem, ale z klasą jako całością.
- Inicjalizowane tylko raz, przed pierwszym uzyskaniem dostępu do pola.

```
class BankAccount {  
    private string accountOwner;  
    private double balance;  
    public static double interestRate = 3.5;  
}
```



Pola statyczne

- Nie trzeba tworzyć instancji klasy, aby uzyskać dostęp do jej pól statycznych.
- Dostęp do nich odbywa się przez nazwę klasy.

```
public class BankAccount {  
    private string accountOwner;  
    private double balance;  
    public static double interestRate = 3.5;  
  
    public void ApplyInterest() {  
        // Dostęp wewnątrz klasy  
        balance += balance * BankAccount.interestRate / 100;  
    }  
}  
  
// Dostęp w kodzie klienta – Main()  
BankAccount.interestRate = 4.5
```

- W **C#** dostęp zawsze przez nazwę klasy: `BankAccount.interestRate`.
- W **Java** i **C++** dozwolony jest również (choć **niezalecany**) dostęp przez instancję:
`account.interestRate`.
- W **C++** dostęp przez klasę używa zasięgu `::` zamiast `.`: `BankAccount::interestRate`.

Inicjalizacja pól statycznych w C++

- W **C++** pola statyczne powinny być inicjalizowane poza klasą:

```
class BankAccount {  
public:  
    static double interestRate; // Deklaracja w klasie  
};  
  
// Inicjalizacja poza klasą  
double BankAccount::interestRate = 3.5;
```

- Wyjątek stanowią pola `inline static` (C++17):

```
class BankAccount {  
public:  
    inline static double interestRate = 3.5; // Inicjalizacja w klasie  
};
```

Konstruktor statyczny

W C#

- Nie przyjmuje parametrów.
- Wywoływany automatycznie przy pierwszym użyciu klasy.
- Służy do inicjalizacji pól statycznych.

```
class BankAccount {  
    public static double InterestRate;  
  
    // Statyczny konstruktor  
    static BankAccount() {  
        InterestRate = 3.5;  
    }  
}
```

W Javie

- Jego odpowiednikiem jest blok `static { }`.

```
class Bank {  
    public static double interestRate;  
  
    // Statyczny blok inicjalizacyjny  
    static {  
        interestRate = 3.5;  
    }  
}
```

Przydają się, gdy inicjalizacja pola statycznego wymaga większej logiki niż pojedyncze wyrażenie.

Pola statyczne – zastosowania

- Przechowywanie globalnych konfiguracji / stałych.

```
class Config {  
    public static readonly string DatabaseUrl = "https://db.example.com";  
}  
Console.WriteLine(Config.DatabaseUrl); // Dostęp bez tworzenia obiektu
```

- Liczniki obiektów, generowanie unikalnych identyfikatorów.

```
class User {  
    private static int nextId = 1; // Statyczne pole przechowujące ID  
    public int Id { get; }  
  
    public User() {  
        Id = nextId++; // Przypisanie unikalnego ID  
    }  
}
```

- Wszędzie tam, gdzie dane mają być współdzielone przez całą aplikację.
- Stanowią lepszą alternatywę dla zmiennych globalnych.

Metody statyczne

- Mogą być wywoływane bez tworzenia instancji klasy.
- **Nie mają dostępu do składowych instancji** (`this` jest niedostępne).
 - Nie możemy sięgać do pól obiektu (bo nie ma obiektu).
 - Nie możemy wywoływać metod instancyjnych (a jedynie inne statyczne).
- Najczęściej wykorzystywane do operacji narzędziowych (*utility methods*).

```
public class MathUtils {  
    public static int square(int x) {  
        return x * x;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        int result = MathUtils.square(5); // Bez tworzenia obiektu  
        System.out.println(result); // Output: 25  
    }  
}
```

- Dostęp do metod statycznych jest taki, jak do pól statycznych.

Klasy statyczne

- W C# można oznaczyć całą klasę jako `static`.
- Taka klasa może zawierać tylko statyczne pola i metody.
- Nie można tworzyć jej instancji.

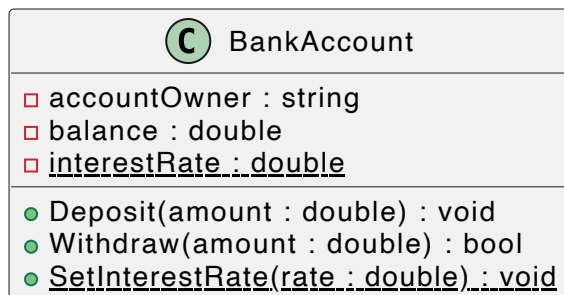
```
static class MathUtils {  
    public static int Square(int x) {  
        return x * x;  
    }  
}
```

// Użycie:

```
int result = MathUtils.Square(5);
```

Składowe statyczne w UML

- Na diagramach klas UML pola i metody statyczne (`static`) są oznaczane podkreśleniem.



elementy statyczne należą
do klasy, nie do obiektu

```
class BankAccount {  
    // Pola instancyjne  
    private string accountOwner;  
    private double balance;  
  
    // Pole statyczne – oprocentowanie  
    private static double interestRate = 3.5;  
  
    public BankAccount(...) { ... }  
  
    // Metody instancyjne  
    public void Deposit(double amount) { ... }  
    public bool Withdraw(double amount) { ... }  
  
    // Metoda statyczna  
    public static void SetInterestRate(double rate) {  
        if (rate > 0) {  
            interestRate = rate;  
        }  
    }  
}
```